

Graphes et Réseaux

420-311 Structures de données

Août 2026

1 Introduction et Représentation

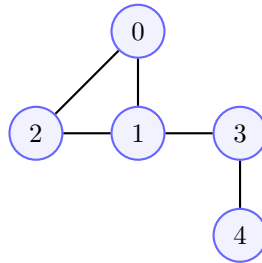
Un **graphe** est une structure de données non linéaire constituée d'un ensemble de sommets (ou nœuds, *vertices*) reliés par des arêtes (ou arcs, *edges*). Contrairement aux arbres, les graphes peuvent contenir des cycles et des interconnexions complexes.

Il existe deux principales manières de représenter un graphe en mémoire :

- **Matrice d'adjacence** : Un tableau à double entrée de taille $|V| \times |V|$ où la case (i, j) contient 1 (ou le poids) s'il y a une arête entre le sommet i et le sommet j , et 0 sinon. Idéal pour les graphes denses.
- **Liste d'adjacence** : Chaque sommet possède une collection (comme une liste ou un dictionnaire) contenant tous ses voisins directs. Idéal pour les graphes clairsemés (*sparse graphs*) et très utilisé en Python via les dictionnaires.

2 Exemples concrets de Représentation (5 sommets)

2.1 1. Graphe non orienté



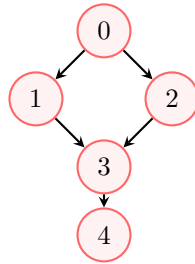
Matrice d'adjacence :

$$M = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

Liste d'adjacence (Python) :

```
graphe_non_oriente = {
    0: [1, 2],
    1: [0, 2, 3],
    2: [0, 1],
    3: [1, 4],
    4: [3]
}
```

2.2 2. Graphe orienté (DAG)



Matrice d'adjacence (Lignes $\rightarrow$ Colonnes) :

$$M = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Liste d'adjacence (Python) :

```
graphe_orienté = {
    0: [1, 2],
    1: [3],
    2: [3],
    3: [4],
    4: []
}
```

3 Types de Graphes et Propriétés

- **Graphe non orienté vs orienté** : Dans un graphe non orienté, les arêtes sont bidirectionnelles. Dans un graphe orienté, les arêtes ont une direction précise (des flèches).
- **Graphe pondéré** : Chaque arête se voit attribuer un poids ou un coût (ex. : distance routière, coût de transaction).
- **DAG (*Directed Acyclic Graph*)** : Un graphe orienté ne contenant aucun cycle. Indispensable pour modéliser des dépendances séquentielles sans boucle infinie.
- **Connexité** : Un graphe non orienté est **connexe** s'il existe un chemin entre toute paire de sommets.
- **Arbre couvrant (*Spanning Tree*)** : Un sous-graphe d'un graphe connexe qui inclut tous les sommets sous forme d'un arbre acyclique.

4 Complexité algorithmique et Matrice d'adjacence

Soit $|V|$ le nombre de sommets et $|E|$ le nombre d'arêtes :

Opération / Caractéristique	Matrice d'adjacence	Liste d'adjacence
Espace mémoire	$O(V ^2)$	$O(V + E)$
Vérifier si (u, v) existe	$O(1)$	$O(deg(u))$
Trouver tous les voisins de u	$O(V)$	$O(deg(u))$
Ajout d'un sommet	$O(V ^2)$	$O(1)$
Ajout d'une arête	$O(1)$	$O(1)$

5 Algorithmes Classiques

- **BFS (*Breadth-First Search* - Parcours en largeur)** : Explore le graphe niveau par niveau en utilisant une file (*queue*). Utilisé pour trouver le plus court chemin dans un graphe non pondéré.
- **DFS (*Depth-First Search* - Parcours en profondeur)** : Explore aussi loin que possible le long de chaque branche en utilisant une pile (*stack*) ou la récursivité. Utilisé pour la détection de cycles et le tri topologique.

6 Applications

- **Réseaux sociaux** : Modélisation des utilisateurs comme des sommets et des amitiés/abonnements comme des arêtes (ex. algorithmes de recommandation d'amis).
- **GPS et Cartographie** : Calcul des itinéraires routiers optimaux (algorithmes de Dijkstra ou A*).
- **Gestion des dépendances (Build systems & DAGs)** : Ordonnancement des tâches de compilation dans des outils comme Make, Gradle ou le chargement de modules.
- **Moteurs de recherche (PageRank)** : Analyse du Web où les pages sont des sommets et les hyperliens sont des arêtes orientées pour déterminer l'autorité des sites.
- **Protocoles de routage Internet** : Optimisation du trafic de données à travers les routeurs du réseau mondial.

7 Lore (Historique)

L'histoire de la théorie des graphes commence en **1736** avec le mathématicien suisse **Leonhard Euler** et le célèbre problème des **ponts de Königsberg**. La ville était traversée par un fleuve comprenant deux îles reliées entre elles et aux rives par sept ponts. Les habitants se demandaient s'il était possible de se promener en empruntant chaque pont une et une seule fois. Euler a modélisé le problème sous forme de sommets (terres) et d'arêtes (ponts), démontrant mathématiquement que c'était impossible, fondant ainsi la topologie et la théorie des graphes moderne.